c5b68357-cfe8-4bd0-b892-f351b1d50cc9

<current>

<request>

You are working on an issue in the 'AzureAD/microsoft-authentication-library-for-objc' repository.

<repository_context>
I've cloned the repository in the directory /home/runner/work/microsoft-authentication-library-for-objc/microsoft-authentication-library-for-objc (not in /tmp/inputs). Always use absolute paths when referring to files in the repository.
</repository_context>

<current_datetime>2026-04-22T03:22:38.262+00:00</current_datetime>

Consider the following problem statement:

<problem_statement>
Show the final content of docs/design/mdm-onboarding-approach-comparison.md on ref dev in AzureAD/microsoft-authentication-library-for-objc. If it doesn't exist or differs, update it to match the finalized design doc discussed in chat (including updated flow diagrams) and open a PR against dev.

The following is the prior conversation context from the user's chat exploration (may be truncated):

User: you can see in the repo MSIDSwitchBrowserResponse and MSIDSwitchBrowserOperation also MSIDAADOAuth COntroller there is comment for PKeyAuth

Assistant: I’m waiting for the agent session to finish analyzing `MSIDSwitchBrowserResponse` / `MSIDSwitchBrowserOperation` and the PKeyAuth-related flow in `MSIDAADOAuthController`; once it completes, I’ll summarize the call chains and update the design doc with the findings and recommendation.

User: @Copilot Accepted Confirmation: Confirm agent session

Assistant: I can’t actually “see” those classes in the repo from here right now, so I can’t truthfully claim I reviewed `MSIDSwitchBrowserResponse`, `MSIDSwitchBrowserOperation`, or the PKeyAuth comment in `MSIDAADOAuthController` in this message.

If you want a real deep analysis (call chain + responsibilities + where the decisions live), please paste one of the following and I’ll analyze it immediately:

1) **Permalinks** to:
- `MSIDSwitchBrowserResponse.*`
- `MSIDSwitchBrowserOperation.*`
- the relevant portion of `MSIDAADOAuthController.*` (the PKeyAuth comment + surrounding code)

or

2) The **code snippets** (even ~150–300 lines around the key methods), especially:
- where `ASWebAuthenticationSession` is created/started and where its callback is handled
- where PKeyAuth is detected and what the “delegate hook” is

Once you share that, I’ll return:
- a precise step-by-step flow for each (switch-browser and PKeyAuth),
- how each maps to “delegate/navigation-time” vs “response-object/operation”,
- and what that implies for your Mobile Onboarding recommendation and doc wording.

User: for pkeyAUth:
```
//
// Copyright (c) Microsoft Corporation.
// All rights reserved.
//
// This code is licensed under the MIT License.
//
// Permission is hereby granted, free of charge, to any person obtaining a copy
// of this software and associated documentation files(the "Software"), to deal
// in the Software without restriction, including without limitation the rights
// to use, copy, modify, merge, publish, distribute, sublicense, and / or sell
```

```
// copies of the Software, and to permit persons to whom the
Software is
// furnished to do so, subject to the following conditions :
//
// The above copyright notice and this permission notice shall
be included in
// all copies or substantial portions of the Software.
//
// THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY
KIND, EXPRESS OR
// IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF
MERCHANTABILITY,
// FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO
EVENT SHALL THE
// AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES
OR OTHER
// LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR
OTHERWISE, ARISING FROM,
// OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN
// THE SOFTWARE.

#import "MSIDSwitchBrowserOperation.h"
#import "MSIDSystemWebviewController.h"
#import "MSIDWebviewResponse.h"
#import "MSIDSwitchBrowserResponse.h"
#import "MSIDWebResponseOperationFactory.h"
#import "MSIDMainThreadUtil.h"
#import "MSIDSwitchBrowserResumeResponse.h"
#import "MSIDWebviewAuthorization.h"
#import "MSIDCertAuthManager.h"
#import "MSIDInteractiveTokenRequestParameters.h"
#import "MSIDConstants.h"
#import "MSIDFlightManager.h"

@interface MSIDSwitchBrowserOperation()

@property (nonatomic) MSIDSwitchBrowserResponse
*switchBrowserResponse;

@end


@implementation MSIDSwitchBrowserOperation

+ (void)load
{
```

```
    [MSIDWebResponseOperationFactory registerOperationClass:self
forResponseClass:MSIDSwitchBrowserResponse.class];
}

- (nullable instancetype)initWithResponse:(MSIDWebviewResponse
*)response
                                    error:(NSError
*__autoreleasing *)error
{
    self = [super initWithResponse:response error:error];
    if (self)
    {
        if (![response
isKindOfClass:MSIDSwitchBrowserResponse.class])
        {
   `
2. Builds `startURL` from:
   - `switchBrowserResponse.actionUri`
   - query params:
     - `code = switchBrowserSessionToken`
     - `redirect_uri = requestParameters.redirectUri`
     - optionally `state`
3. Launches system auth via **manager**:
   - `MSIDCertAuthManager startWithURL:parentController:…
completionBlock:`
   - (this is effectively where `ASWebAuthenticationSession` is
launched)
4. On callback:
   - converts `callbackURL` → new response via:
     - `[webRequestConfiguration
responseWithResultURL:callbackURL
factory:oauthFactory.webviewFactory …]`
   - sets `response.parentResponse = self.switchBrowserResponse`
   - calls `webviewResponseCompletionBlock(response, nil)`

**Resume operation**
1. Validates state (flight-gated)
2. Sets `webRequestConfiguration.startURL = actionUri`
3. Injects auth into headers:
   - `Authorization: Bearer <switchBrowserSessionToken>`
4. Creates an embedded webview from factory and calls:
   - `[MSIDWebviewAuthorization startSessionWithWebView:…
completionHandler:]`

**Takeaway**
This is a clean example of **response-object-based
orchestration** working well when:
```

```
- the event is a *semantic server instruction* (“switch to
system web auth, then come back”)
- there’s a natural “operation” to execute
- the callback becomes a normal webview response again

---

### 2) PKeyAuth pattern = **navigation-time interception**
(delegate-style decision at the webview boundary)
From the screenshot snippet in `MSIDAADOAuthController`
(WKNavigationDelegate path):

**What it does**
- Detects PKeyAuth challenge by URL prefix:
  - `if ([requestURLString hasPrefix:[kMSIDPKeyAuthUrn
lowercaseString]])`
- Cancels navigation immediately:
  - `decisionHandler(WKNavigationActionPolicyCancel);`
- Handles challenge out-of-band:
  - `[MSIDPKeyAuthHandler handleChallenge: …
completionHandler:^(NSURLRequest *challengeResponse, NSError
*error) { … }]`
- If successful, it resumes the embedded flow by loading the
returned request:
  - `[self loadRequest:challengeResponse];`

**Takeaway**
This is a canonical example of the “**delegate/navigation-action
orchestration**” pattern:
- the webview boundary detects a special condition
- it cancels navigation
- it runs custom logic (network/crypto/challenge)
- it resumes by loading a new request back into the same
embedded webview session

---

## What this implies for Mobile Onboarding (your two
requirements)

### Requirement A: special redirect URLs (`msauth://enroll`,
`msauth://compliance`, `msauth://enrollment_complete`)
These behave much closer to **PKeyAuth** than switch-browser:

- They are **mid-flight navigations** that instruct the client
to do work (BRT acquisition once per redirect) and then load a
constructed next URL with required params/headers.
```

```
- That is exactly “cancel/replace navigation with a new request”
behavior.

So the best-fitting implementation model is the **navigation-
time interception pattern** (PKeyAuth-like).

### Requirement B: response headers for telemetry + header-
driven ASWebAuth handoff
This is also webview-boundary logic:
- headers are available at navigation-response time
- the decision “launch ASWebAuth now” must happen at that moment

You *can* model the actual “launch ASWebAuth” part using the
**switch-browser style response/operation** *internally*, but
the *trigger point* still naturally lives at the navigation
delegate boundary.

---

## Recommendation (now grounded in current code patterns)
**Use navigation-time interception (delegate-style) as the
primary orchestration for Mobile Onboarding**, because it
matches the existing PKeyAuth pattern: cancel navigation → do
required work (BRT acquisition / header evaluation) → load the
next request.

Optionally, for the ASWebAuth launch itself, you can reuse the
**operation-style encapsulation** (like switch-browser) as an
implementation detail, but do not force the entire onboarding
redirect flow through “completeAuth → factory → response object”
because it’s not a terminal completion event.

---

## One clarifying question (to make the doc precise)
For the header-driven ASWebAuth handoff: when you launch
`ASWebAuthenticationSession`, do you need to **resume the *same*
embedded session** (same `WKWebView` instance), or is it
acceptable to treat the callback as a new “result URL” into the
normal response parsing pipeline?

(Your earlier description suggests “resume embedded flow”, but
confirming affects the diagram and boundary rules.)

User: @Copilot Accepted Confirmation: Confirm agent session

Assistant: Yes — with what you shared, I’ve now done a **deep
```

```
analysis of the existing patterns** (at least for the concrete
code you pasted for switch-browser, and the concrete PKeyAuth
interception snippet you showed).

### Switch-browser (response-object + operation) pattern — what
it proves
From your `MSIDSwitchBrowserOperation` /
`MSIDSwitchBrowserResumeOperation` snippet:

- The flow is **explicitly modeled as a typed response →
executed by a registered operation**.
- The “operation” is where the system webview
(`ASWebAuthenticationSession`-style) is launched (via the cert
auth manager), and the **callback URL is translated back into a
normal webview response** using
`responseWithResultURL:factory:...`.
- Resume is a separate operation that:
  - restores `startURL`,
  - injects `Authorization: Bearer <session token>` header,
  - creates a webview via the factory and resumes the embedded
flow through the standard `MSIDWebviewAuthorization` pipeline.

**Meaning:** response-object/operation orchestration works great
when the event is a **semantic instruction** like “switch
context to system web auth, come back, then continue.”

### PKeyAuth (navigation-time interception) pattern — what it
proves
From the `MSIDAADOAuthController` snippet you showed:

- It detects a special condition **during navigation-action** by
matching the request URL prefix (`kMSIDPKeyAuthUrn`).
- It **cancels navigation immediately**.
- It executes out-of-band handling (`MSIDPKeyAuthHandler
handleChallenge:...`).
- It **resumes the embedded session** by loading a returned
`NSURLRequest` via `loadRequest:`.

**Meaning:** delegate/navigation-time orchestration is the
established pattern in this repo for “special URL means do work
now, then replace navigation.”

---

## What this means for Mobile Onboarding: my recommendation (now
grounded in these patterns)
```

```
### My recommendation remains:
**Use the PKeyAuth-style navigation-time interception (delegate
pattern) as the primary orchestration for Mobile Onboarding.**

### Why (in repo terms)
- `msauth://enroll` and `msauth://compliance` behave like
**PKeyAuth**: they are **mid-flight navigation instructions**
that require:
  - intercept/cancel,
  - do BRT acquisition (once per redirect),
  - construct the real next URL from query params + add required
query params/headers,
  - and then load the request.
- The header-driven ASWebAuth trigger is also most naturally
decided at the **navigation-response boundary**, because that's
where headers exist.

### Where to reuse the switch-browser idea
Even if the *trigger* is delegate/navigation-time, you can still
reuse the **switch-browser encapsulation** idea for the actual
"launch system web auth and convert callback URL into a
response," because that's a proven, testable unit in this repo.

So the clean boundary is:

- **Delegate/navigation-time owns detection + decision**
(special redirect URLs, response headers, telemetry decisions).
- **Operation/response object can own execution of ASWebAuth
launch** (optional reuse), returning a callback URL that the
controller feeds back into the normal flow.

---

## One key question to finalize the design doc correctly
When ASWebAuth completes and you get the callback URL, do you
need to resume **the same embedded webview session** (same
`WKWebView` / same controller instance), or can you treat the
callback as a "result URL" that you parse and then proceed
without reloading it in the embedded webview?

That single detail changes:
- whether the doc should recommend "load callback URL into
embedded view" vs "parse callback and produce response," and
- how similar the handoff should be to
`MSIDSwitchBrowserOperation`'s `responseWithResultURL:`
behavior.
```

```
User: resume the same embedded webview session

Assistant: With “**resume the same embedded webview session**”
confirmed, my recommendation becomes even stronger and more
specific:

## Updated recommendation (final)
Use **delegate / navigation-time orchestration (Approach A)** as
the primary architecture for Mobile Onboarding, because it
naturally supports **cancel/replace navigation** and **resuming
the same `WKWebView` instance** after doing off-webview work
(BRT acquisition, header-driven ASWebAuth handoff).

### Why the “resume same embedded session” requirement is
decisive
Resuming the same embedded session implies you must preserve:
- the **same `WKWebView` instance lifecycle** (navigation state,
back/forward list)
- cookies/session continuity as observed by that webview (or at
least keep the same webview container and continuation
semantics)
- an uninterrupted “in-progress” web auth session (you’re not
completing and restarting auth; you’re continuing it)

This is exactly how the repo’s **PKeyAuth** pattern works:
- intercept in navigation delegate
- `decisionHandler(WKNavigationActionPolicyCancel)`
- run handler
- then `loadRequest:` back into the **same webview**

Trying to force this through “completeAuth → factory response →
controller operation” increases the risk that you accidentally
model the event as a terminal outcome and lose “same webview
continuation” semantics.

### Best-practice boundary (to avoid dual-path complexity)
- **Delegate path owns detection + stateful continuation**:
  - `msauth://enroll`, `msauth://compliance`: cancel navigation
→ BRT acquisition once per redirect → build final URL with
required headers/query params → load in same webview
  - response headers: telemetry + detect ASWebAuth handoff →
temporarily suspend webview UI → launch ASWebAuth → on callback
load callback URL into same webview
- **Response objects** only represent *semantic end states*
(e.g., `msauth://enrollment_complete`, final success/error), not
“continue-the-flow” redirects.
```

If you still want to reuse the switch-browser modularity, you can encapsulate *only* “launch ASWebAuth and return callback URL” as a helper/operation, but the orchestration and resumption should remain in the embedded-controller delegate so it always returns to the same `WKWebView`.

If you paste the final doc content you want (or let me generate it in-chat), I can produce a complete `mdm-onboarding-approach-comparison.md` section that explicitly calls out this “resume same embedded webview session” requirement and ties it to the PKeyAuth precedent and the switch-browser mechanics.

User: no but can you paste the final doc here

</problem_statement>

Implement the necessary changes to the repository so that the requirements specified in the <problem_statement> are met.

A **CI/Build Failure** has been mentioned in the problem statement, make sure to follow your instructions for addressing CI/Build Failures.